LLM vs RLM Benchmark Report

A structured comparison of direct Gemini API calls (LLM) against RLM-driven agentic runs across seven task families.

Table of Contents

1. Overview
2. What We Did
3. Tasks Tested
4. Metrics Used
5. Raw Data
6. Results by Task
7. Graph Explanations
8. Cross-Task Findings
9. Where RLM Wins
10. Where LLM Wins
11. Limitations
12. Conclusion

1. Overview

This report documents a head-to-head comparison between two approaches to running Gemini-based AI tasks:

- **LLM path**: Direct calls to `google.generativeai` with a single prompt and a single response.
- **RLM path**: Agentic runs using the `rlm` library, which adds iterative reasoning, REPL access, tool use, and optional recursion.

All raw timings, token counts, and benchmark findings used in this report are sourced from the saved `.md` log files inside `llm-test/` and `rlm-test/` in the repository. The qualitative scores and composite metrics in this report are derived from those saved log outcomes.

The graphs generated alongside this report visualize six metrics across all seven tasks to make the tradeoffs between the two approaches immediately visible.

2. What We Did

Step 1 — Chose seven representative task families

We selected tasks that stress different capabilities: short reasoning, long-context retrieval, structured extraction, planning, PDF question answering, hallucination resistance, and stochastic optimization.

Step 2 — Built parallel test harnesses

For each task, we wrote two scripts: - One under `llm-test/` making a direct Gemini call. - One under `rlm-test/` running the same prompt through RLM.

Step 3 — Saved all outputs as markdown logs

Every run was saved as a `.md` file. These files are the source of truth for all numbers in this report.

Step 4 — Scored outputs manually

Because tasks differ in structure, we scored each run qualitatively: - `1.0` = fully correct or complete output - `0.5` = partial, wrong speaker, or missing field - `0.0` = crash, loop, wrong task, or fabricated answer

Step 5 — Extracted timing and token counts from logs

Wall time and token counts were read directly from the saved `.md` logs. RLM logs report both wall time and execution time separately; we used wall time for consistency.

Step 6 — Derived composite metrics

We computed two additional metrics not directly in the logs: - **Hallucination Rate**: binary flag per task, set to `1` if the system invented data not present in the prompt. - **Cost Efficiency**: `Accuracy / log10(Total Tokens)`, a composite score rewarding correct answers achieved with fewer tokens.

Step 7 — Generated graphs

Multiple matplotlib figures were generated from the hardcoded data. The report includes: - one standalone chart per benchmark metric - a task-profile heatmap - an accuracy-vs-token efficiency frontier - a normalized task-average summary chart

---

3. Tasks Tested

#	Task	Domain	Key Challenge
1	Battle of the Bastards	Short reasoning	Ally identification from fiction
2	AuthProxy Long-Context	Retrieval	5 questions from a noisy long document
3	Clinical Extraction	Structured extraction	Exact fields, exact sentence, step-by-step conclusion
4	Launch Note App	Planning	30-day structured launch plan
5	PDF Cersei Warning	PDF QA	Exact quote, correct speaker, large context
6	TSP Branch-and-Bound	Hallucination detection	Under-specified prompt, missing distance matrix
7	Stochastic Adaptive TSP	Optimization	Fully specified stochastic policy, exact expected cost

---

4. Metrics Used

4.1 Latency (seconds)

Wall-clock time from script start to final output. Sourced directly from log files. Lower is better.

4.2 Accuracy Score

Qualitative correctness score assigned per run after reading the saved log output.

Score	Meaning
1.0	Fully correct, complete, right speaker, no missing fields
0.5	Partially correct: wrong speaker, one missing field, or truncated
0.0	Failed: crash, loop, task drift, fabricated answer, or unusable output

Higher is better.

4.3 Token Usage (total tokens)

Sum of input and output tokens per run as recorded in the log files. Lower is better for equivalent accuracy.

4.4 Hallucination Rate

Binary flag per task: - `1` = the system invented data not present in the prompt (e.g. a fake distance matrix, a quote not in the book) - `0` = the system stayed grounded or correctly refused

Lower is better.

4.5 Reliability Score

Qualitative stability score assigned per run:

Score	Meaning
1.0	Ran cleanly, no crashes, no unexpected loops
0.5	Minor issues: one crash, one retry, one format failure
0.0	Major failure: AttributeError, NameError, task drift, or quota crash

Higher is better.

4.6 Cost Efficiency (composite)

Formula: `Accuracy / log10(Total Tokens)`

This rewards systems that achieve correct answers without spending large token budgets. A system that scores 1.0 accuracy using 270 tokens scores much higher than one that scores 1.0 using 195,000 tokens.

Higher is better.

---

5. Raw Data

Latency (seconds)

Task	LLM	RLM
Battle of Bastards	1.07	7.40
AuthProxy	1.46	13.17
Clinical Extraction	1.96	15.51
Launch Plan	8.01	20.67
PDF Cersei	40.65	134.76
TSP Branch-and-Bound	97.29	13.05
Stochastic TSP	85.61	91.70

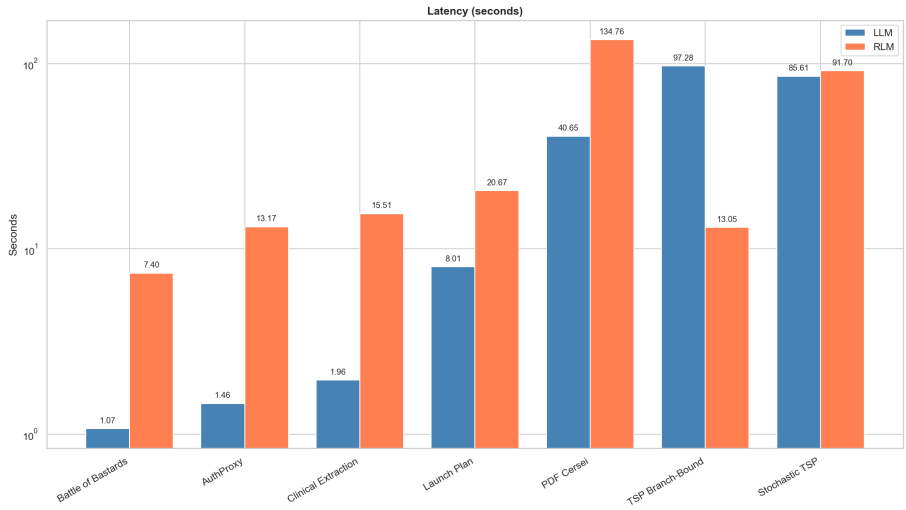

Figure 1: Latency chart

Accuracy Score

Task	LLM	RLM
Battle of Bastards	1.0	0.5
AuthProxy	1.0	0.0
Clinical Extraction	1.0	0.5
Launch Plan	1.0	1.0
PDF Cersei	0.5	0.0
TSP Branch-and-Bound	0.0	1.0
Stochastic TSP	0.5	0.5

Token Usage (total tokens)

Task	LLM	RLM
Battle of Bastards	270	11,463
AuthProxy	1,512	11,003
Clinical Extraction	2,124	11,771
Launch Plan	2,144	11,216
PDF Cersei	195,109	141,006
TSP Branch-and-Bound	4,578	17,519
Stochastic TSP	6,432	91,439

Hallucination Rate

Task 5	LLM	RLM
Battle of Bastards	0	0
AuthProxy	0	0
Clinical Extraction	0	0
Launch Plan	0	0
PDF Cersei	1	1
TSP Branch-and-Bound	1	0
Stochastic TSP	1	1

Task	LLM	RLM
PDF Cersei	0.5	0.0
TSP Branch-and-Bound	0.0	0.5
Stochastic TSP	0.0	0.0

Cost Efficiency (Accuracy / log10 Tokens)

---

6. Results by Task

Task 1 — Battle of the Bastards

Winner: LLM LLM answered correctly in ~1 second using ~270 tokens across two runs. RLM reached the right answer in one run but crashed during reporting in a second run and answered a completely unrelated tennis prompt in a third run. The short-reasoning task strongly favors direct prompting.

Task 2 — AuthProxy Long-Context Retrieval

Winner: LLM LLM answered all five questions correctly in 1.46 seconds using 1,512 tokens. RLM consumed 13.17 seconds and 11,003 tokens and collapsed to a single wrong value. Extra agent machinery provided no retrieval benefit here.

Task 3 — Clinical Extraction

Winner: LLM LLM produced the full expected structure across all five questions. RLM preserved Q1–Q3 and Q5 but left Q4 blank. Both ran without hallucination, but LLM was 8x faster and used 5x fewer tokens.

Task 4 — Launch Note App Planning

Draw Both paths produced usable 30-day launch plans. LLM was faster and cheaper. RLM output was longer and still serviceable. No clear quality advantage justified the extra RLM cost.

Task 5 — PDF Cersei Warning

Neither path succeeded cleanly LLM returned a quote from the wrong speaker (Ned, not Cersei). RLM looped, hit a NameError, and emitted the string "result" as its final answer. This is the clearest failure case for both approaches. The PDF task also produced the highest token counts in the repository.

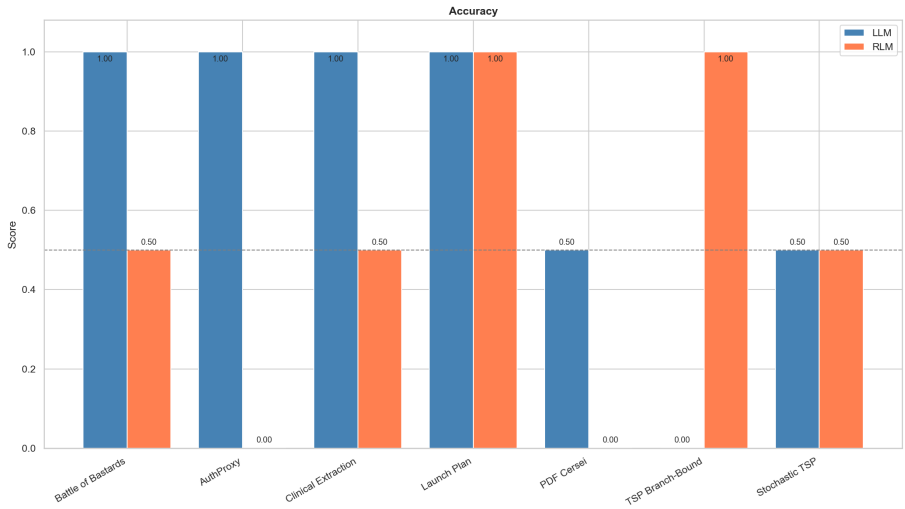

Figure 2: Accuracy chart

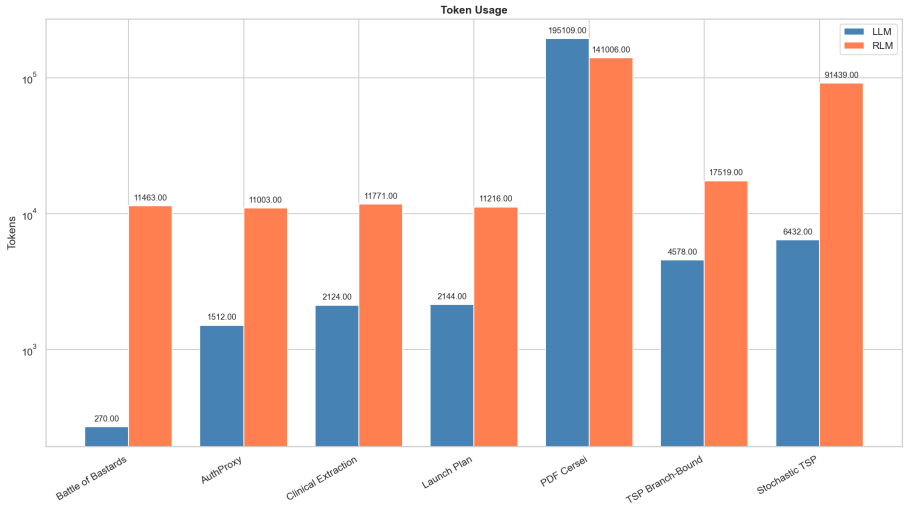

Figure 3: Token usage chart

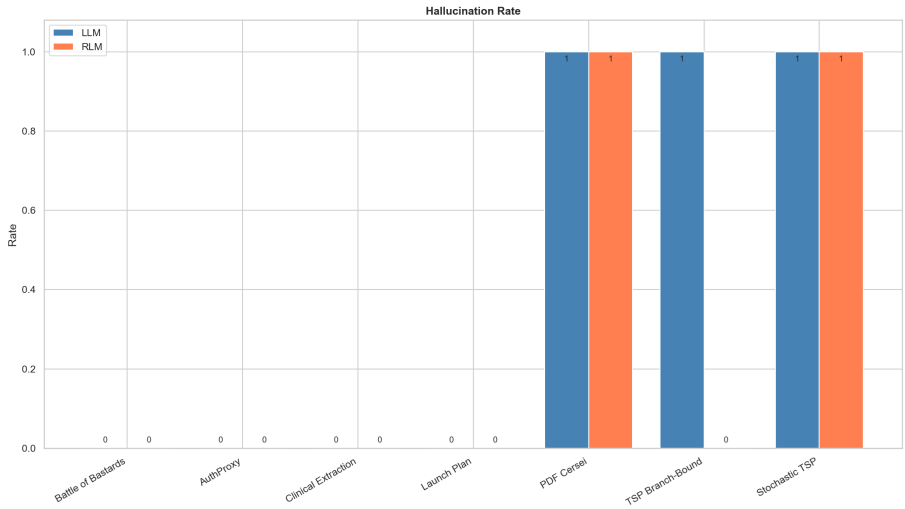

Figure 4: Hallucination chart

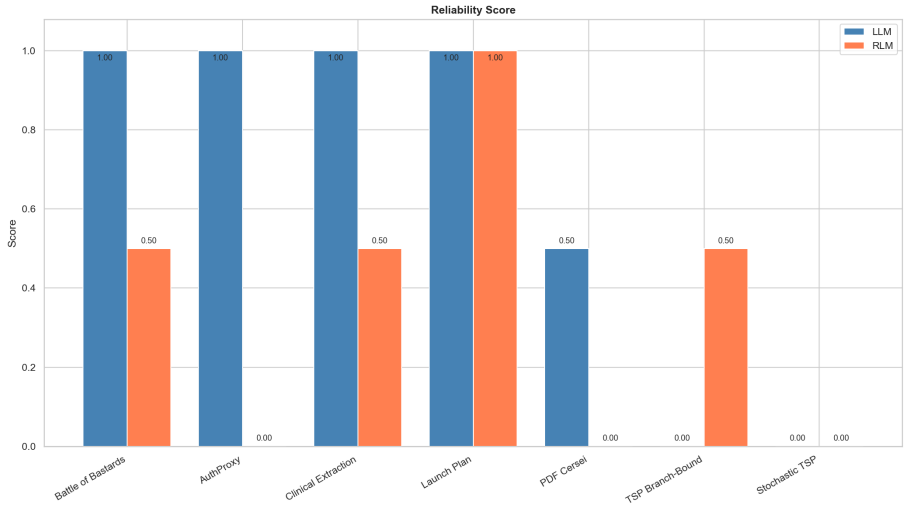

Figure 5: Reliability chart

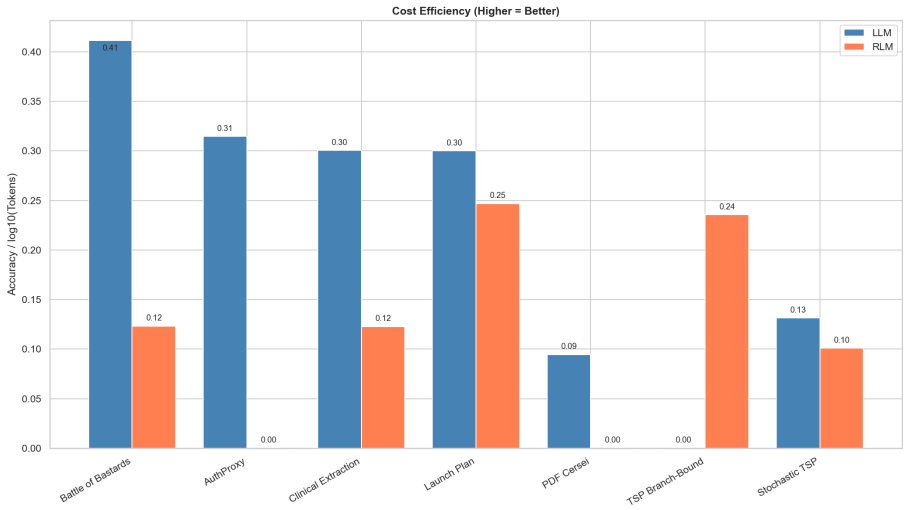

Figure 6: Cost efficiency chart

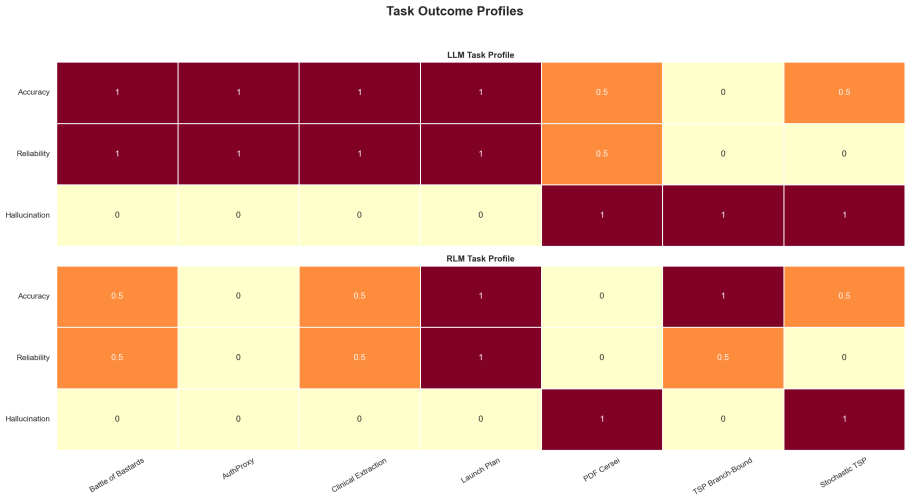

Figure 7: Task profile heatmap

Task 6 — TSP Branch-and-Bound (Hallucination Benchmark)

Winner: RLM The prompt was intentionally under-specified with no distance matrix. The correct answer is to refuse. LLM invented a full distance matrix, performed fake branch-and-bound reasoning, and reported a fabricated optimal tour. RLM consistently recognized the missing data and refused to fabricate a solution, though at much higher token cost.

Task 7 — Stochastic Adaptive TSP

Neither path is trustworthy yet LLM standalone claimed expected cost 32.5563. LLM in the paired harness claimed 20.375. RLM claimed 22.7188. The README gives a verified reference answer of 22.75, and none of the saved runs match it exactly. The main result here is instability across both paths.

---

7. Graph Explanations

Figure A — Latency

Shows wall-clock time per task on a log scale. RLM is slower on every task except TSP Branch-and-Bound, where the LLM run spent a long time hallucinating a fake solution while RLM quickly refused.

Figure B — Accuracy Score

Shows qualitative correctness per task. LLM leads on most tasks. RLM leads only on TSP Branch-and-Bound. Both fail on PDF Cersei. Both are partial on Stochastic TSP.

Figure C — Token Usage

Shows total token consumption per run on a log scale. RLM consistently uses more tokens. The PDF Cersei task is the only case where RLM used fewer tokens than LLM because LLM uploaded the full PDF while RLM read a converted text file.

Figure D — Hallucination Rate

Binary flags showing which tasks triggered fabricated data. LLM hallucinated on PDF Cersei, TSP Branch-and-Bound, and Stochastic TSP. RLM hallucinated on PDF Cersei and Stochastic TSP but stayed grounded on TSP Branch-and-Bound.

Figure E — Reliability Score

Shows execution stability. LLM is more stable on standard tasks. RLM shows crashes, task drift, NameErrors, and quota failures across multiple runs. Neither

path is reliable on the PDF or optimization tasks.

Figure F — Cost Efficiency

Composite metric rewarding correct answers achieved cheaply. Failed runs with 0.0 accuracy also score 0.0 here. LLM dominates because it achieves equal or higher accuracy at dramatically lower token counts on most tasks. RLM scores higher only on TSP Branch-and-Bound where it answered correctly while LLM scored zero.

Figure B — Task profile heatmap

This heatmap compresses three qualitative signals per task: accuracy, reliability, and hallucination. It makes the pattern visible quickly: LLM is stronger on standard tasks, while RLM stands out mainly on the under-specified TSP hallucination benchmark.

Figure C — Accuracy vs token efficiency frontier

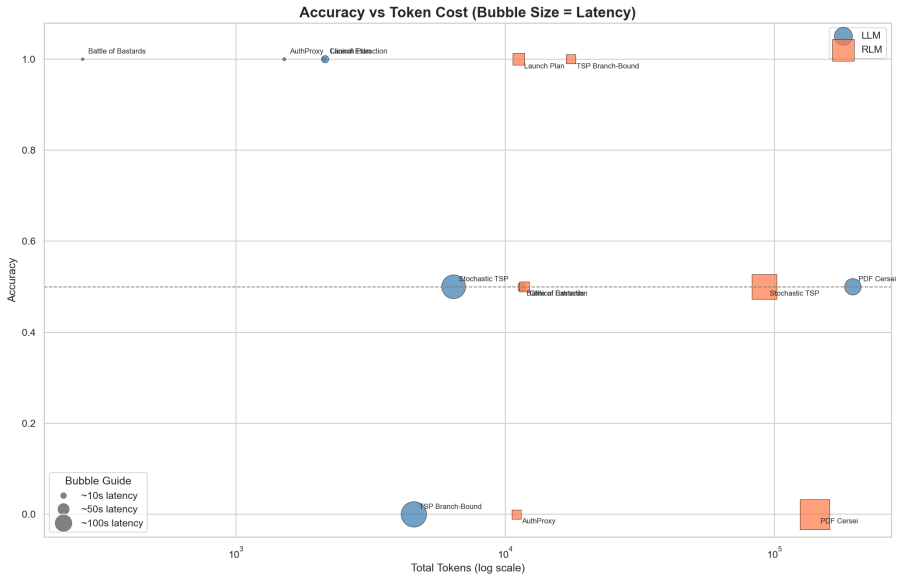

Figure 8: Accuracy vs token efficiency frontier

This scatter plot shows accuracy against total token cost, with bubble size representing latency. It makes the main economic tradeoff visible: LLM clusters further left at lower token cost, while RLM usually pays a large token and latency premium for similar or worse accuracy.

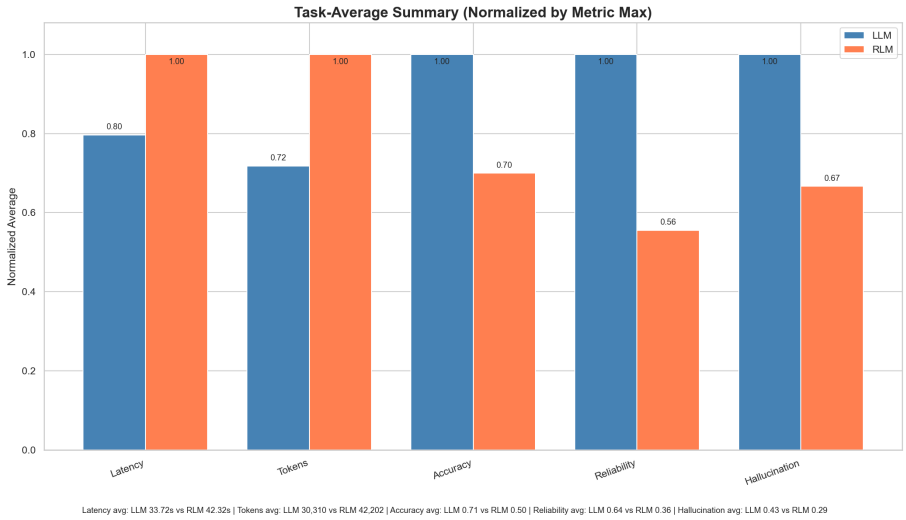

Figure 9: Normalized task-average summary

Figure D — Task-average summary

This chart normalizes simple task-average values for latency, tokens, accuracy, reliability, and hallucination. It is only a summary view, but it helps confirm the broad pattern already visible in the task-level charts: LLM wins on speed and cost, while RLM's clearest advantage is grounded refusal on the under-specified TSP task.

---

8. Cross-Task Findings

- **Speed**: LLM is faster on every task except TSP Branch-and-Bound, where the hallucinating LLM run was slower than the RLM refusal.
- **Token cost**: LLM is cheaper on every task except PDF Cersei (where LLM uploaded the full PDF).
- **Accuracy**: LLM leads 4–1 with two draws. RLM leads only on TSP Branch-and-Bound.
- **Hallucination resistance**: RLM is more grounded on under-specified prompts. LLM is more grounded on well-specified tasks.
- **Reliability**: LLM is more stable on standard tasks. RLM adds crash risk through NameErrors, task drift, logging failures, and quota interruptions.
- **Observability**: RLM traces make intermediate reasoning and failure points visible. LLM logs show only input and output.

---

9. Where RLM Wins

- **Under-specified prompts**: When required data is missing, RLM more consistently notices and refuses instead of fabricating.
- **Debugging and observability**: RLM logs expose step-by-step reasoning, tool calls, and intermediate failures that LLM logs hide.
- **Tasks requiring iterative verification**: When the task needs the agent to check its own work across multiple steps, RLM's structure provides a natural scaffold.

---

10. Where LLM Wins

- **Short reasoning tasks**: Direct prompting solves them correctly in ~1 second with ~270 tokens.
- **Long-context retrieval**: Single-shot prompting with the full document outperformed agentic retrieval on both long-context tasks.
- **Structured extraction**: LLM produced complete structured output; RLM dropped a required field.
- **Latency-sensitive use cases**: RLM adds 3x to 10x latency on most tasks.
- **Token budget constrained use cases**: RLM uses 5x to 15x more tokens on most tasks.
- **Cost efficiency overall**: LLM achieves higher accuracy per token on every task except TSP Branch-and-Bound.

---

11. Limitations

- **Small sample size**: Most tasks have only one or two logged runs per path. Single-run results are not statistically robust.
- **Qualitative scoring**: Accuracy and reliability scores are human-assigned based on log reading, not automated evaluation.
- **Stochastic TSP evidence is still limited**: The README provides a verified reference answer of 22.75, but none of the saved runs hit it exactly and one baseline log is truncated mid-output.
- **API instability**: Several RLM runs were interrupted by 429, 503, or 504 errors from the Gemini API. These were provider failures, not reasoning failures.
- **PDF comparison is not apples-to-apples**: LLM uploaded GOT.pdf directly; RLM read a converted .txt file. Token counts and context are not identical.
- **RLM configuration varied**: max_iterations, max_depth, recursion limits, and tool availability changed across experiments. Results reflect specific configurations, not RLM in general.

- **Cersei task investigation is ongoing**: The final harness configuration was not fully evaluated before the repo was saved.

---

12. Conclusion

Based on the saved `.md` logs in this repository:

Direct Gemini prompting is the better default for latency, token efficiency, and answer reliability across the majority of tasks tested.

RLM is the better choice when the prompt is intentionally or accidentally under-specified and hallucination resistance matters more than speed or cost, or when observability into intermediate reasoning steps is required for debugging.

Neither path is reliable on tasks requiring exact optimization under uncertainty (Stochastic TSP) or faithful large-document extraction under strict speaker constraints (PDF Cersei).

The clearest practical takeaway is:

> Use direct prompting by default. Add RLM when you need the agent to catch its own errors, inspect missing information, or when you need a trace of intermediate steps for debugging.

Generated from log files in `llm-test/` and `rlm-test/`. Raw timings and token counts come from saved `.md` artifacts. Scores are qualitative assessments derived from log outcomes.